otris software

@otr-documents/otrpackage - Handbuch

Weitergabe und Vervielfältigung dieser Publikation oder von Teilen daraus sind, zu welchem Zweck und in welcher Form auch immer, ohne die ausdrückliche schriftliche Genehmigung durch die otris software AG nicht gestattet.

In dieser Publikation enthaltene Informationen können ohne vorherige Ankündigung geändert werden.

Alle in dieser Publikation aufgeführten Wort- und Bildmarken sind Eigentum der entsprechenden Hersteller.

Änderungen in der Software sind vorbehalten. Die in dieser Publikation enthaltenen Informationen stellen keinerlei Verpflichtung seitens des Verkäufers dar.

Aus Gründen der besseren Lesbarkeit wird auf eine geschlechtsneutrale Differenzierung verzichtet. Entsprechende Begriffe gelten im Sinne der Gleichbehandlung grundsätzlich für alle Geschlechter. Die verkürzte Sprachform beinhaltet keine Wertung. Dies gilt für die Begriffe in dieser Publikation ebenso wie für die Software selbst.

© Copyright 2026 otris software AG. Alle Rechte vorbehalten.

Inhaltsverzeichnis

otr-package ... 4
- Zweck ... 4
- Funktionen ... 4
- Auf einem Blick ... 4
- Begriffe ... 4

Getting Started ... 5
- Projektinitialisierung ... 5
- Paket-Spezifikation (PACKAGE-Script) ... 5
- Update-Script ... 7
- Bauen und Installieren ... 9

Feature Walkthrough ... 9
- Neue Version erstellen ... 10
- Abhängigkeiten definieren ... 10
- Checks und Hooks ... 12
- Installations-Konfigurationangaben ... 14
- Mappen-Ändern-Strategien ... 17
- Löschen veralteter Scripte ... 17
- Admin-Center-Integration ... 18
- ADP-Optimierung ... 20
- Versionsgetreue Installation ... 20
- Vereinfachter Backport ... 22

Paketverwaltung im Admin Center ... 23
- Liste der Installationsvorgänge ... 23
- Installation starten ... 23
- Versionshistorie ... 24

Konfiguration ... 25
- Installationsnutzer ... 25
- Build-Konfiguration ... 25
- Deploy-Konfiguration ... 29

Changelog ... 31
- 0.7.5 ... 31
- v0.7.4 ... 32
- v0.7.3 ... 32
- v0.7.2 ... 32
- v0.7.1 ... 32
- v0.7.0 ... 32
- v0.6.0 ... 33
- v0.5.1 ... 33
- v0.5.0 ... 33
- v0.4.9 ... 33
- v0.4.8 ... 33
- v0.4.7 ... 34

v0.4.6 34
v0.4.5 34
v0.4.4 34
v0.4.3 34
v0.4.2 34
v0.4.1 34
v0.4.0 35
v0.3.1 35
v0.3.0 35
v0.2.0 36
v0.1.36 36
v0.1.36 36
v0.1.35 37
v0.1.34 37
v0.1.33 37
v0.1.32 37
v0.1.31 37
v0.1.30 37
v0.1.29 38
v0.1.28 38
v0.1.27 38
v0.1.26 38
v0.1.25 38
v0.1.24 39
v0.1.23 39
v0.1.22 41

otr-package

Zweck

Das `otr-package` ist ein Tool von documentsOS, das zur Erstellung und Verwaltung von Lösungspaketen dient.

Funktionen

Mit dem `otr-package` können

- Installationspakete für Lösungen erstellt werden.
- Diese Pakete in einen documentsOS-Mandanten installiert werden.

Auf einem Blick

- Getting Started: Schritt-für-Schritt-Anleitung für das Aufsetzen eines neuen documentsOS-Projektes.
- Feature Walkthrough: Detaillierte Erläuterungen und Beispiele zu verschiedenen Features, einschließlich Abhängigkeiten, Checks und Hooks, Erstellung neuer Versionen sowie ADP-Optimierung.
- Paketverwaltung: Erläuterung, wie Pakete im Admin-Center verwaltet werden.
- Konfiguration: Erläuterung der Konfiguration des Bauens und Installieren von Paketen.
- API: Eine umfassende Beschreibung der `otr-package`-API.

Begriffe

Zentrale Begriffe, die in den Beschreibungen verwendet werden:

- *Application Deployment Package* (ADP): Ein Installationspaket für eine documentsOS-Lösung oder ein Modul, vergleichbar mit einem MSI-Paket unter Windows oder einem Debian-Paket unter Linux. Ein ADP enthält die SDPs aller Versionen und Abhängigkeiten, die für die Installation der Lösung erforderlich sind.
- *Single Distribution Package* (SDP): Ein Installationspaket für eine spezifische Version, das alle Artefakte einer Lösungsversions oder einer Modulversion enthält, wie z.B. Skripte und Workflows.
- *PACKAGE-Script*: Ein spezielles Skript, das ein Paket beschreibt, einschließlich Installationsanweisungen und Metadaten wie Versionsnummer und Beschreibungen.
- *Update-Script*: Ein spezielles Skript, das während der Installation ausgeführt wird, um Änderungen an einem Mandanten vorzunehmen, z.B. das Anlegen eines neuen Mappentyps.

Getting Started

Das Getting Started für documentsOS zeigt wie ein vollständiges Projekt aus einer Vorlage erstellt wird. Es enthält die folgenden Schritte:

1. Initialisieren eines Projekts aus einer Vorlage
2. Anpassen der Paket-Spezifikation
3. Erstellen eines Update-Scripts
4. Bauen und Installieren des Pakets

Für eine ausführlichere Erläuterung einzelner Features siehe den Feature Walkthrough.

Projektinitialisierung

Der schnellste Weg, um ein Projekt anzulegen, ist die Verwendung des Befehls `documents init`:

```
mkdir my-project
cd my-project
documents init
```

Dadurch wird das Projekt und mit seiner Ordnerstruktur initialisiert:

```
❯ documents init
√ Target directory the project:  ... C:/Users/VeitJahns/Documents/Projekte/my-project
Install template...
...
```

Beim Aufsetzen der Vorlage werden verschiedene Verzeichnisse und Dateien angelegt - entweder als Vorlagen oder vorbereitete Konfigurationen für verwendete Tools. Zur Erläuterung aller Verzeichnisse und Dateien ist eine Datei `README.md` beigefügt. Zudem wird für das Projekt ein Git-Repository initialisiert.

Paket-Spezifikation (PACKAGE-Script)

Mit der Vorlage werden alle notwendigen Ordner und Init-Scripte erzeugt. Im Ordner `src/jscript/init.cat` liegen die Scripte, die für die Installation des Paket notwendig sind. Zentral ist dabei das PACKAGE-Script (Namensschema `${PROJECT_NAME}_PACKAGE.js`):

```
// #crypt

/**
 * Paket-Spezifikation für my-project
 */

const { declarePackage } = require("otr-package");

module.exports = declarePackage({
    version: "@versionNumber@",
    name: "my-project",
    description: "",
    dependencies: [],
    updateScripts: {
        "@versionNumber@": [
            "my-project.xml",
            "issue_1"
        ]
    }
});
```

Folgende Minimalangaben müssen gemacht werden:

- Die beschreibenden Eigenschaften `version`, `name`, `description` sind für die Paketverwaltung erforderlich. Der Platzhalter `@versionNumber@` wird beim Bauen durch die aktuelle Versionsnummer aus der `package.json` ersetzt.
- Die Eigenschaft `dependencies` definiert Abhängigkeiten (siehe Abschnitt zu Abhängigkeiten).
- Die Eigenschaft `updateScripts` definiert die Update-Schritte für alle Versionen und dient somit als Installationsanleitung für das Paket. Bei mehreren Versionen kann dies beispielsweise wie folgt aussehen:

```
updateScripts: {
  "1.0.0": [
    "my-project.xml", // Importiere my-project.xml aus Version v1.0.0 dem Paket
    "issue_1" // Führe Update-Script "issue_1" aus
  ],
  "@versionNumber@": [
    "my-project.xml", // Importiere my-project.xml aus der aktuellen Version
    "issue_2" // Führe Update-Script "issue_2" aus
  ]
}
```

Für alle weiteren verfügbaren Properties sowie deren Zweck und Funktionsweise siehe die Dokumentation der Paket-Deklaration.

Update-Script

Einzelne Änderungen wie das Anlegen und Ändern eines Mappentyps werden in Update-Scripten definiert. Diese liegen im Ordner `src/jscript/update.cat`, und weisen folgenden Aufbau auf:

```
// #crypt

/**
 * Vorlage für ein Update-Script
 */

const { makeUpdateScript } = require("otr-package");

module.exports = makeUpdateScript({

    execute(context) {
        // ...
    },

    submitFileChanges: {
        // ...
    },

    postHandling(context) {
        // ...
    }
});
```

Ein Update-Skript besteht aus drei Hauptbestandteilen:

- `execute`: Die eigentliche Update-Funktion, um z.B. Mappentypen anzulegen oder zu ändern, welche mit otrUpgrade implementiert werden können.
- `submitFileChanges`: Angabe der Mappentypen, auf die "Mappen ändern" ausgeführt werden soll, um Änderung auf bestehende Mappen zu übertragen.

  ```
  submitFileChanges: {
    Mappentyp: true
  }
  ```

- `postHandling`: Funktion, die nach Abschluss von `execute` und `submitFileChanges` ausgeführt, um z.B. Datenmigrationen auszuführen.

Siehe hierzu auch die API-Dokumentation.

Bauen und Installieren

Der Build-Prozess umfasst verschiedene Schritte. Diese sind bereits als Script-Task in der `package.json` vordefiniert. Auf der Konsole kann daher einfach der `build`-Task ausgeführt werden:

```
npm run build
```

Es werden der Reihe nach die XMLs, das SDPs und das ADP gebaut. Die XMLs liegen im Ordner `build` und die SDPs und ADPs im Ordner `deploy`.

Die Installation kann mittels des Script-Tasks `deploy` erfolgen:

```
# Installation auf einem lokalem System
npm run deploy -- --principal $name
```

Feature Walkthrough

Dieses Dokument bietet eine kurze Übersicht über die einzelnen Features von `otr-package`.

Die Erläuterungen basieren auf der Standardkonfiguration des Projekts. Bei Abweichungen können zusätzliche Anpassungen erforderlich sein.

Dieses Dokument beschreibt folgende Features:

- *Neue Version*: Erstellen einer neuen Version
- *Abhängigkeiten*: Einbinden von Abhängigkeiten
- *Check und Hooks*: Definieren von Checks und Hooks während des Installationsprozesses
- *Installations-Konfigurationangaben*: Definieren von Konfigurationen, die während der Installation vorgenommen werden können, z.B. Verbindungsdaten zu weiteren Datenbanken.
- *Mappen-Ändern-Modus*: Definieren, wann während der Installation eines Pakets das Mappen-Ändern ausgeführt wird.
- *Löschen veralteter Scripte*: Automatisches Löschen von veralteten Scripten bei der Installation eines Pakets.
- *Admin-Center-Integration*: Definieren der Einbindung des Pakets in das Admin-Center.
- *Anpassen der Build- und Deploy-Konfiguration*: Erweitertes Konfigurieren für das Bauen und Installieren der `otr-package`-Artefakte.
- *ADP-Optimierung*: Optimierung der Erstellung der Installationspakete.
- *Versionsgetreue Installation*: Installation in exakter Reihenfolge der Versionen der Hauptlösung.
- *Vereinfachter Backport*: Korrektur bereits ausgelieferter Pakete.

Hinweis: Alle Code-Beispiele sind in TypeScript verfasst, können funktionieren aber auch analog in JavaScript.

Neue Version erstellen

Für eine neue Version eines Pakets, ist das PACKAGE-Script zu ergänzen:

- Ergänzen einer neuen `updateScripts`-Liste für die neue Version.

```
// ...
module.export = declarePackage({
    // ...
    updateScripts: {
        "1.0.0": [ // Bereits veröffentlichte Version
            "my-project.xml",
            "issue_1"
        ],
        "1.1.0": [ // Neue, geplannte Version
            "my-project.xml",
            "issue_2",
            "issue_3"
            // ...
        ]
    },
    // ...
});
```

- Generell wird empfohlen die Update-Scripte der einzelnen Versionen in einem separatem Ordner abzulegen. Bezogen auf das zuvor genannte Beispiel wären dies
    - `issue_1` in `src/jscript/update.cat/1.0.0/`
    - `issue_2` und `issue_3` in `src/jscript/update.cat/1.1.0/`

Siehe hierzu auch die API-Dokumentation.

Abhängigkeiten definieren

Um abhängige Pakete zusammen mit einem Paket zu installieren, können diese in der Paketdeklaration unter `dependencies` angegeben werden:

```
// ...
module.export = declarePackage({
    // ...
    dependencies: [
        "otrsign",
        "otrlanguage"
    ],
    // ...
});
```

An dieser Stelle ist der Name des Pakets einzutragen. Bei Scriptlib-Paketen (z. B. otrSign) sind keine weiteren Schritte erforderlich. Bei anderen Paketen (z. B. otrLanguage) hängt es davon ab, ob sie aus einer Registry geladen werden oder im Projekt selbst mit vorhanden sind.

Abhängigkeiten werden vor dem eigentlichen Paket und in der Reihenfolge installiert, wie sie in `dependencies` angegeben sind.

Siehe hierzu auch die API-Dokumentation.

Abhängigkeiten aus Registries

Um Abhängigkeiten aus Registries zu verwenden, müssen diese im Projekt installiert werden:

```
npm install --save @otr-documents/otrlanguage
```

Weitere Anpassungen sind in der Regel nicht notwendig. Beim Bauen werden die notwendigen Build-Artefakte der Abhängigkeiten von `documents` bzw. `otr-build` automatisch berücksichtigt, um das Gesamtpaket zu bauen.

Es können aktuell aber nur die Abhängigkeiten verwendet werden, die in den Scopes `@otr-documents` und `@documents-components`.

Abhängigkeiten in lokalen Projekt

Projektabhängigkeiten können auch direkt im Projekt abgelegt werden. Diese müssen im Verzeichnis `${PROJECT_ROOT}/dependencies` mit folgender Struktur abgelegt werden:

```
dependencies
 +- ${DEPENDENCY_NAME}
    +- deploy
       +- ${DEPENDENCY_NAME}_V1.0.0.sdp
```

Beim Bauvorgang muss zusätzlich der Parameter `--dev` angegeben werden. Dadurch werden alle SDPs aus dem Verzeichnis `dependencies` in den ADP-Bauprozess einbezogen.

Checks und Hooks

Prerequisites

Während des Installationsprozesses können an verschiedenen Stellen Funktionen ausgeführt werden, beispielsweise zur Überprüfung von Installationsvoraussetzungen. Die einfachste Methode ist die Verwendung der `prerequisites`-Funktion. Diese wird unmittelbar nach dem Import der Paketdeklaration ausgeführt.

Wirft diese Funktion eine Exception oder gibt ein Array mit Fehlermeldungen zurück, wird der Installationsvorgang abgebrochen. In dem Fall werden die Paketspezifikationen zurückgerollt. Da zu diesem Zeitpunkt lediglich die Paketspezifikationen importiert wurden, befindet sich der Mandant anschließend wieder im ursprünglichen Zustand.

```
// ...
module.export = declarePackage({
    // ...
    prerequisites() {
        const condition = ...;
        if (!condition) {
            return [ "Prerequisites not fulfilled!" ];
        }
        return [];
    },
    // ...
});
```

Siehe hierzu auch die API-Dokumentation.

Pre-Update-Scripte

Eine weitere Möglichkeit bieten die `preUpdateScripts`. Dabei handelt es sich um Update-Scripte, die vor der Installation jeder einzelnen Paketversion ausgeführt werden. Als Update-Scripte ermöglichen sie Fehlerprüfungen und Korrekturen im Zielmandanten, um z.B. eine fehlerfreie Installation sicherzustellen.

Die Pre-Update-Scripte werden vor der Installation jeder einzelnen Version ausgeführt. Bei zwei Versionen werden die `preUpdateScripts` beispielsweise vor der Installation von Version 1 und erneut vor der Installation von Version 2 ausgeführt. Es kann sich dabei um eine beliebige Anzahl von Update-Scripten handeln:

```
// ...
module.export = declarePackage({
    // ...
    preUpdateScripts: [
        "pre-update-script-1",
        "pre-update-script-2"
    ],
    // ...
});
```

Die Scripte müssen (aktuell) im Package Deployment Descriptor enthalten sein, d.h., im Ordner `src/jscript/init.cat` liegen.

Siehe hierzu auch die API-Dokumentation.

Globale Pre- und Post-Update-Scripte

Als dritte Möglichkeit gibt es globale Pre- und Post-Update-Scripte, die sogenannten `globalScripts`. Die globalen Pre-Update-Scripte werden zu Beginn ausgeführt, noch vor der Installation der ersten Version oder der ersten Abhängigkeit. Die Post-Update-Scripte werden abschließend nach erfolgreicher Installation der letzten Version ausgeführt.

Wie bei den `preUpdateScripts` handelt es sich dabei um Update-Scripte. Der besondere Vorteil der `globalScripts` besteht darin, dass zwischen einer Erstinstallation (`new`) und einem Update (`update`) unterschieden werden kann:

```
// ...
module.export = declarePackage({
    // ...
    globalScripts: {
        new: {
            pre: [
                "pre-update-script-1",
                "pre-update-script-2"
            ],
            post: [
                "post-update-script-1"
            ]
        },
        update: {
            pre: [
                "pre-update-script-3",
            ],
            post: [
                "post-update-script-1"
            ]
        }
    }
    // ...
});
```

Die Scripte müssen (aktuell) im Package Deployment Descriptor enthalten sein, d.h., im Ordner `src/jscript/init.cat` liegen.

Siehe hierzu auch die API-Dokumentation.

Installations-Konfigurationangaben

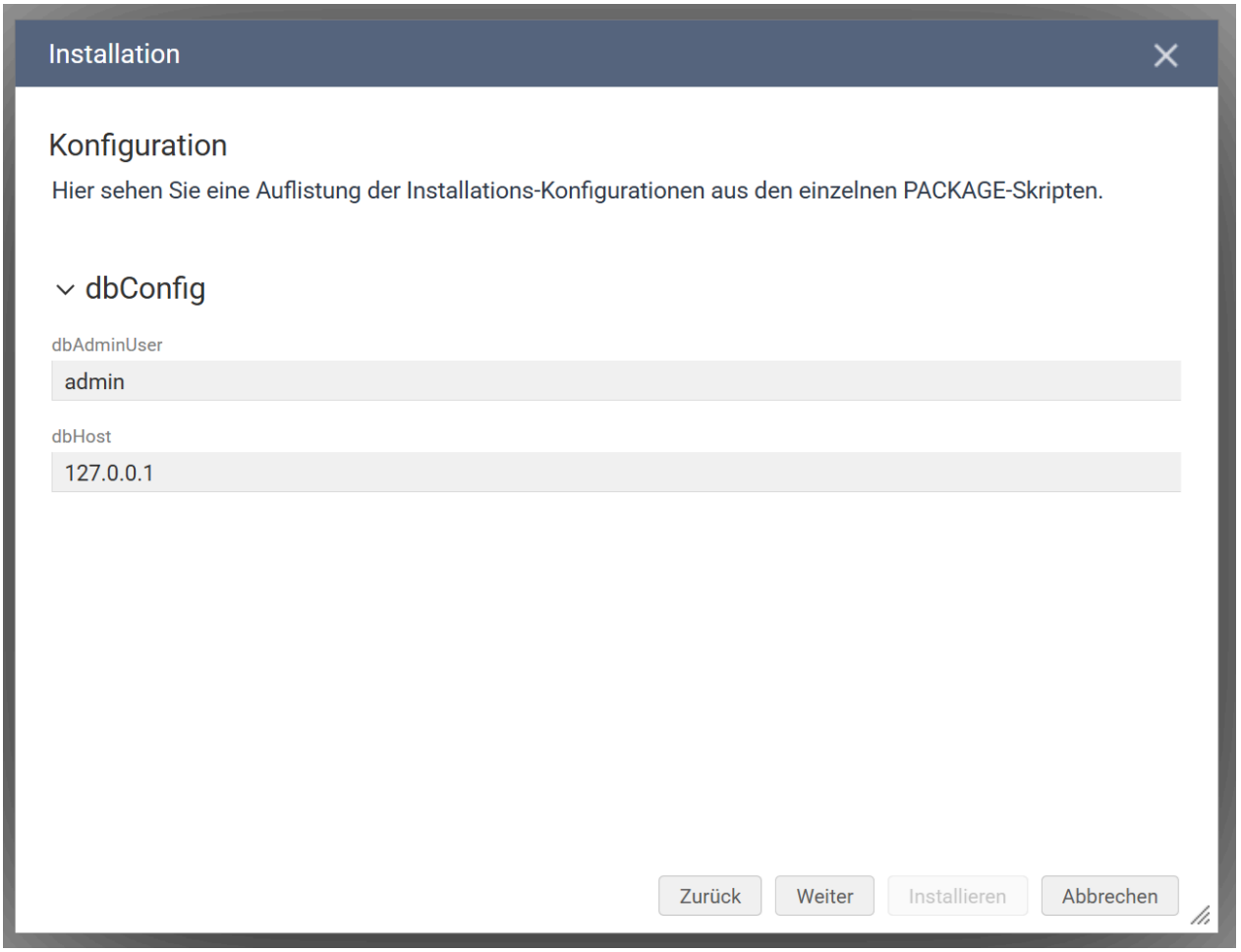
Dem Installationsprozess können Konfigurationsdaten übergeben werden, die während der Installation ausgelesen und verwendet werden. Beispiele hierfür sind Zugangsdaten zu externen Datenbanken oder andere systemrelevante Parameter.

Die Definition der verfügbaren Konfigurationsparameter erfolgt über `installationConfig`:

```
{
  "installationConfig": {
    "dbHost": {
      "type": "string",
      "description": "Datenbank-Hostname",
      "required": true
    },
    "dbPort": {
      "type": "number",
      "description": "Datenbank-Port",
      "default": 5432
    }
  }
}
```

```js
// ...
module.export = declarePackage({
    // ...
    installationConfig: {
        allVersions: true, // Gilt für alle Versionen
        configs: {
            dbConfig: {
                dbAdminUser: "admin", // Default-Werte
                dbHost: "127.0.0.1" // Default-Werte
            }
        }
    },
    // ...
});
```

Vor der Installation fordert der Assistent im Admin-Center die Eingabe von `dbAdminUser` und `dbHost` an:



Auf diese Konfigurationseinstellungen kann in den Update-Scripten wie folgt zugegriffen werden:

```
// ...
module.export = makeUpdateScript({
    execute(context: PackageContext) {
        const configs = context.getInstallationConfig("dbConfig");
        const dbAdminUser = configs.dbAdminUser;
        const dbHost = configs.dbHost;
        // ...
    },
    // ...
});
```

Siehe hierzu auch die API-Dokumentation.

Mappen-Ändern-Strategien

Standardverhalten

Standardmäßig wird nach der Installation jedes SDP (d.h. nach der Installation jeder Version der Lösung oder Abhängigkeit) ein Mappen-Ändern durchgeführt. Dies erhöht zwar die Robustheit des Installationsvorgangs gegenüber Abbrüchen, kann jedoch bei großen Systemen mit vielen Mappen zu längeren Laufzeiten führen, da Mappen-Ändern für die gleichen Mappentypen mehrfach während der Installation ausgeführt werden kann.

Optimiertes Verhalten bei globalem Mappen-Ändern

Durch die Verwendung des Mappen-Ändern-Moduls `global` kann die Mappen-Änderung gebündelt werden, sodass sie nur einmal während der gesamten Installation ausgeführt wird. Dadurch verschiebt sich die Ausführung aller Post-Handling-Prozesse auf den Zeitpunkt nach der Mappen-Änderung:

```
module.export = declarePackage({
    // ..
    submitFileChangeMode: "global",
    // ...
});
```

Alternativ kann der Modus auch per CLI mittels der Option `--submit-file-change-mode` oder per Umgebungsvariable `SUBMIT_FILE_CHANGE_MODE` bei der Installation erzwungen werden:

```
documents deploy --submit-file-change-mode global ...

set SUBMIT_FILE_CHANGE_MODE="global"
documents deploy ...

# Bei otr-build-Variante
otr-build deploy --submit-file-change-mode global ...

set SUBMIT_FILE_CHANGE_MODE="global"
otr-build deploy ...
```

Löschen veralteter Scripte

Während der Paketinstallation können veraltete Skripte automatisch aus dem Mandanten entfernt werden. Dies betrifft insbesondere:

- Skripte, die für die aktuelle Server-Version nicht mehr benötigt werden

- Durch neue Funktionen ersetzte Skripte
- Nicht mehr unterstützte Legacy-Skripte

Der Löschvorgang wird durch die Server-Version gesteuert und folgt dieser Logik:

1. *Versionsprüfung*: Vor dem Löschen wird die Server-Version geprüft
2. *Bedingte Ausführung*: Löschung erfolgt nur bei Erfüllung der Versionsbedingung
3. *Protokollierung*: Alle Löschvorgänge werden dokumentiert

Im folgenden Beispiel wird das Script `otrPerfmon` immer dann gelöscht, wenn das Paket auf einer 6.0.0 oder höher installiert wird. Wird das Paket auf einer älteren Server-Version installiert, dann wird das genannte Script nicht gelöscht.

```
module.export = declarePackage({
    // ..
    scriptBlacklist: [{
        sinceServerVersion: 2501,
            scriptBlacklist: [
                "otrPerfMon"
            ]
    }]
    // ...
});
```

Siehe hierzu auch die API-Dokumentation.

Admin-Center-Integration

Pakete lassen sich in das Admin-Center einbinden, so dass sie nach der Installation dort angezeigt und von dort konfiguriert werden bzw. werden können. Dazu ist in der Paket-Deklaration der `type: "extension"` zu setzen. Dann können weitere Angaben für die Integration in das Admin-Center vorgenommen werden:

- Metadaten zur Beschreibung des Pakets (ergonomischer Titel, Icon, Beschreibung)
- Funktionen zur Integration in das Admin-Center zum De-/Aktivieren, Verrechtung usw. für das De-/Aktivieren kann auf eine Default-Implementierung zurückgegriffen werden, die in `otrPkgModuleTools` bereitgestellt wird.

Siehe das folgende Beispiel:

```
const { declarePackage } = require("otr-package");
const { activate, deactivate } = require("otrPkgModuleTools");

module.export = declarePackage({
    // ..

    // Meta data
    type: "extension",
    title: "Admin Center Example",
    icon: "mdi:mdi-file-cabinet",
    description: "A package that can be configured in the Admin Center.",

    // Admin center functions
    isActive: () => {
        return true; // Package is always active
    },
    activate: (options) => {
        // Do some own stuff
        // ...
        // Activate the package using the default implementation
        activate("PackageName");
    },
    deactivate: (options) => {
        // Do some own stuff
        // ...
        // Deactivate the package using the default implementation
        deactivate("PackageName");
    },
    roles: () => {
        return [{
            name: "exampleRole",
            title: "Example Role",
            description: "This role is an example role"
        }];
    }
    // ...
});
```

Siehe hierzu auch die API-Dokumentation.

ADP-Optimierung

Folgende Schalter sorgen für eine Optimierung der erstellten ADPs. Für zukünftige Versionen kann diese Funktionen aber Default-Verhalten werden, so dass dies nicht mehr angegeben werden muss.

Entfernen unnötiger SDPs

Beim Erstellen der ADPs werden derzeit alle im Ordner `deploy` gefundenen SDPs der Vorgängerversionen und aller Abhängigkeiten bzw. deren Versionen in das ADP eingefügt, auch denn wenn Sie bei der Installation gar nicht verwendet werden.

Der Schalter `--reduce-size` sorgt dafür, dass diese unnötigen SDPs entfernt werden. Da dies aber Abhängig vom PACKAGE-Script ist, wird hierfür einmal das ADP installiert und ausgewertet, welche SDPs tatsächliche installiert wurden. Daher ist für diese Funktion ein Documents-Server notwendig, dessen Verbindungsdaten ebenfalls gegeben werden müssen, wenn sie von den Default-Werten abweichen.

```
documents build --reduce-size --host localhost ...

# Bei otr-build-Variante
otr-build build-adp --reduce-size --host localhost ...
```

Ignorieren der SDPs von dev-Abhängigkeiten

Mittels `--sdp-dep-check` prüft `otr-build` beim Erstellen des ADPs, ob gefundene SDPs zu dev-Abhängigkeiten gehören. Ist das der Fall, werden diese beim Bauen des ADPs ignoriert.

```
documents build --sdp-dep-check ...

# Bei otr-build-Variante
otr-build build-adp --sdp-dep-check ...
```

Versionsgetreue Installation

Verwendung

Um die versionsgetreue Installation zu verwenden, müssen in den SDP-Paketen der Hauptlösung die Ausführungspläne zum Zeitpunkt der Veröffentlichung enthalten sein. Durch Bauen des ADPs mit dem Schalter `--reduce-size` oder `--full` werden diese von `otr-build` automatisch mit in die relevanten SDPs eingefügt:

```
otr-build build-adp --full ...
otr-build build-adp --reduce-size ...
```

Die Ausführungspläne werden bei der Installation nicht automatisch ausgewertet, sondern dies ist zu aktivieren. Entweder durch

- Setzen der Mandanteneigenschaft `$PACKAGE_INSTALL_PINNED_VERSION_ENABLED=true`
- oder Setzen der Umgebungsvariable `PACKAGE_INSTALL_PINNED_VERSION_ENABLED=true`

Erläuterung

Der Unterschied zwischen der normalen und der versionsgetreuen Installation besteht in der Reihenfolge, wie die einzelnen Pakete installiert werden. Das soll an folgendem Beispiel erläutert werden:

```
otr-sample v1.0.0
  +- otr-dep v1.0.0
    +- otr-sub-dep v1.0.0
otr-sample v1.1.0
  +- otr-dep v1.1.0
```

Bei der normalen Installation ist die Installationsreihenfolge:

```
1. otr-sub-dep v1.0.0
2. otr-dep v1.0.0
3. otr-dep v1.1.0
4. otr-sample v1.0.0
5. otr-sample v1.1.0
```

Es werden erst alle Versionen der Abhängigkeiten installiert, und anschließend alle Versionen der Hauptlösung. Dabei wird `otr-dep v1.1.0` vor `otr-sample v1.0.0` installiert. In dieser Version wurde `otr-dep v1.1.0` aber noch nicht verwendet.

In der Regel führt das nicht zu Problemen, insbesondere wenn die neueren Versionen abwärtskompatibel sind. Ist dies aber nicht der Fall, kann dies zu Problemen führen, wenn `otr-sample v1.0.0` installiert wird.

Mit der versionsgetreuen Installation werden erst alle Abhängigkeiten bis zu der Version installiert, die zum Zeitpunkt der jeweiligen Version der Hauptversion veröffentlicht wurden. In dem Beispiel ist dann die Reihenfolge wie folgt:

```
1. otr-sub-dep v1.0.0
2. otr-dep v1.0.0
3. otr-sample v1.0.0
4. otr-dep v1.1.0
5. otr-sample v1.1.0
```

Dadurch wird die Lösung exakt in der Reihenfolge installiert, als ob die beiden Versionen der Hauptlösung separat installiert wurden, also erst `otr-sample v1.0.0` und dann `otr-sample v1.1.0`.

Nachteil dieses Vorgehens ist jedoch, dass der Installation länger dauert. Bei jeder Installation einer Version der Hauptlösung wird selbst im globalen Mappen-Ändern-Modus das Mappen-Ändern und die Post-Handlings ausgeführt. Bei dem Beispiel 2-mal, nach `otr-sample v1.0.0` und nach `otr-sample v1.1.0`.

Vereinfachter Backport

Ein ADP enthält immer die SDPs der Vorgängerversionen zum Zeitpunkt der Auslieferung. Enthält ein solches SDPs ein fehlerhaftes Script, was bereits bei der Installation zu Problemen führt, kann es nicht durch ein korrigiertes ausgetauscht werden.

Um solche Scripte zu korrigieren, kann eine `backport.zip` dem ADP hinzugefügt werden. Aktuell muss diese händisch erstellt und händisch dem ADP hinzugefügt werden:

- Pro Paket und Version muss die `backport.zip` einen Ordner enthalten, in dem eine XML-Export-Datei mit dem korrigiertem Script enthalten ist. Z.B. für das Paket otr-sample v1.0.0 und v1.2.0 sieht die Struktur so aus

  ```
  otr-sample_V1.0.0
    +- otr-sample.xml
  otr-sample_V1.2.0
    +- PackageDeploymentDescriptor.xml
  ```

  Bei v1.0.0 wird die reguläre XML-Datei mit den Scripten von otr-sample überschrieben, bei v1.2.0 der Package Deployment Descriptor der v1.2.0.
- Die `backport.zip` muss in der Wurzel des ADPs neusten Version (hier v1.3.0) liegen:

  ```
  - backport.zip
  - otr-sample_V1.0.0.sdp
  - otr-sample_V1.1.0.sdp
  - otr-sample_V1.2.0.sdp
  - otr-sample_V1.3.0.sdp
  ```

Durch das Entpacken der `backport.zip` werden die jeweiligen XML-Dateien in dem Ordner für die v1.0.0 und die v1.2.0 überschrieben, bevor sie in den Mandanten importiert werden.

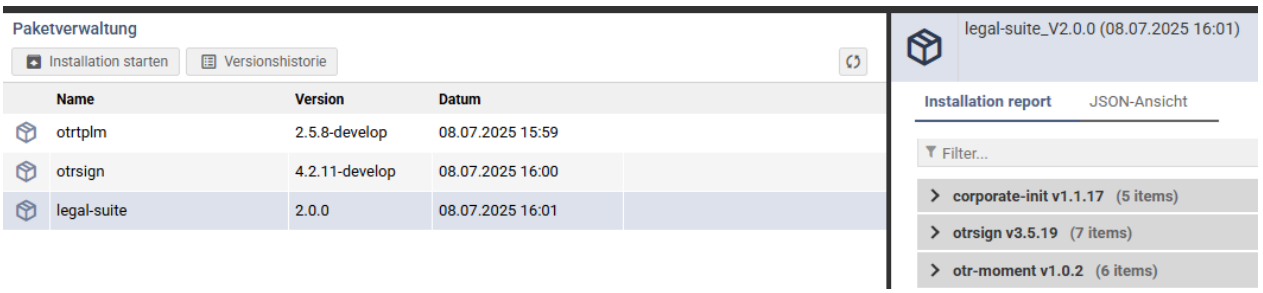
Paketverwaltung im Admin Center

Die Paketverwaltung im Admin Center umfasst folgenden Funktionen:

- *Liste der Installationsvorgänge*: Auflistung aller aiseführten Installationsvorgänge.
- *Installation starten*: Öffnet den Assistent zum Installieren eines Lösungspaket.
- *Versionshistorie:* Zeigt die Versionen der installierten Versionen an.

Liste der Installationsvorgänge

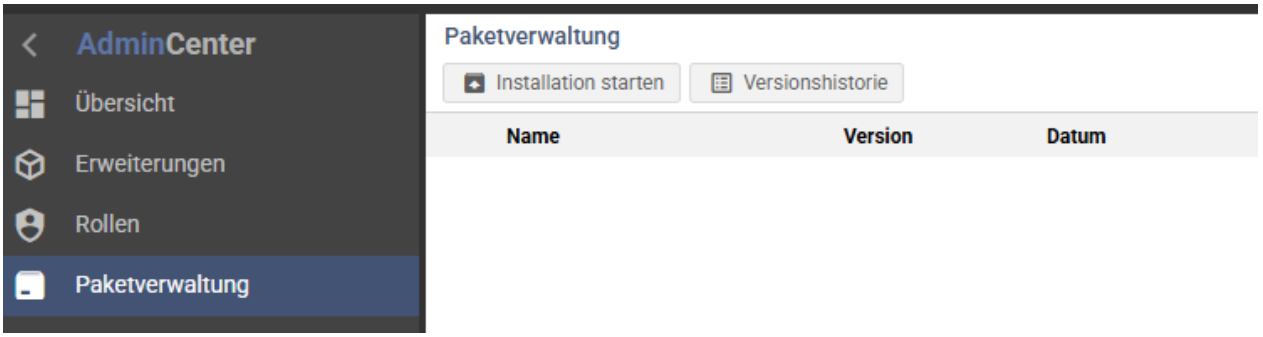
In dieser Liste werden alle Installationsvorgänge mit Informationen darüber, was und wann etwas installiert wurde, aufgeführt. Durch Anklicken eines Eintrags wird der Installationsbericht geöffnet, der weitere Details zum Installationsvorgang anzeigt.



Installation starten

Die Aktion "Installation starten" öffnet einen Assistenten, der durch die Installation eines Lösungspakets führt. Der Assistent umfasst die folgenden Schritte:

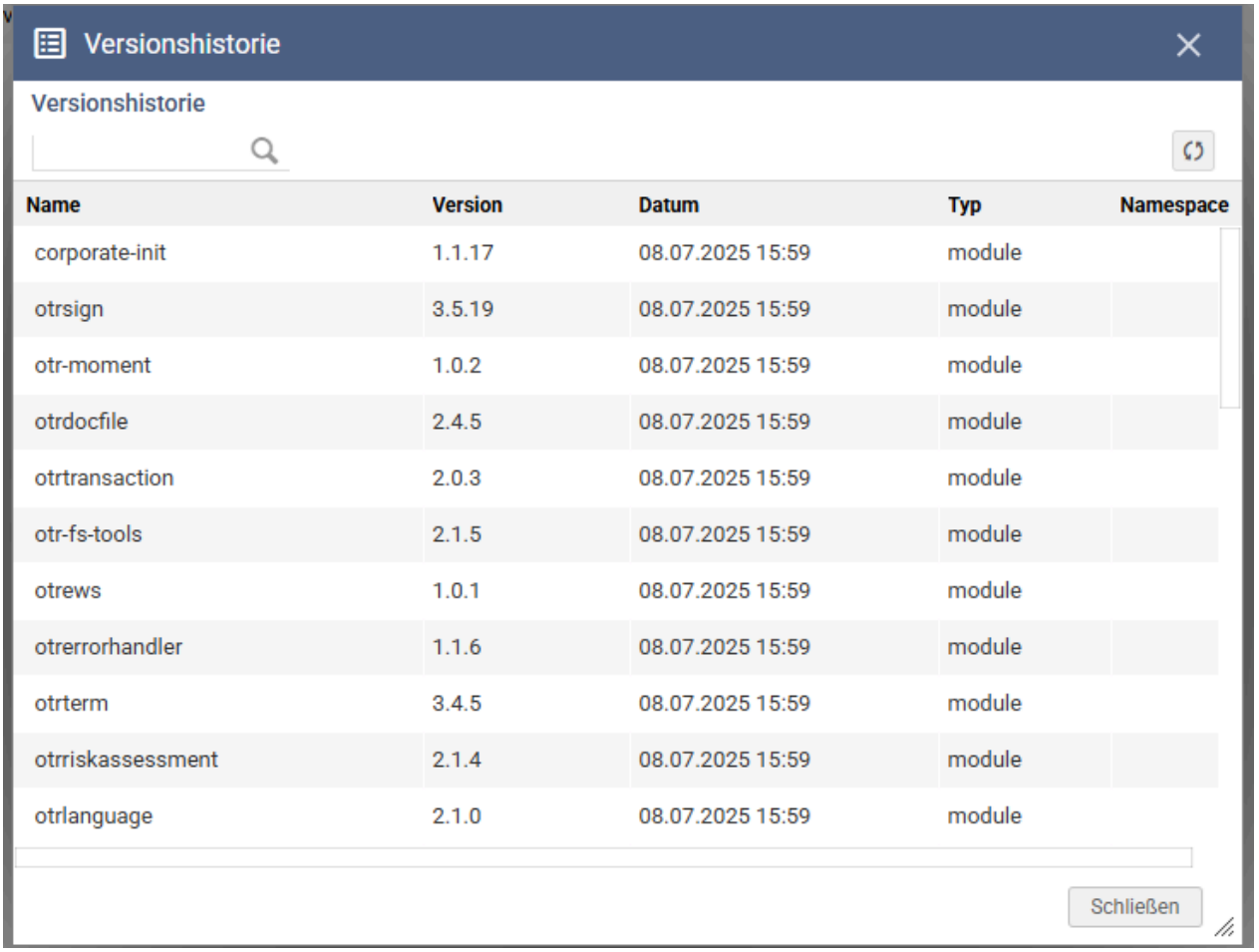
- **Paket auswählen**: Auswahl und Hochladen des Lösungspakets. In der Regel werden hier ADPs hochgeladen. Es können jedoch auch SPDs und XMLs hochgeladen werden. Deren Installation wird ebenfalls in der Liste der Installationsvorgänge anzeigt.
- **Konfigurationsdaten eingeben:** Konfigurationen für den Installationsprozess, wenn die im Paket definiert sind. Ansonsten wird der Schritt übersprungen.
- **Anzeige des Ausführungsplan:** Zeigte die Pakete an, die installiert werden sollen.



Der Assistent führt durch die Installation und startet zum Abschluss die Installation, die nebenläufig im Hintergrund ausgeführt wird.

Versionshistorie

Die Versionshistorie zeigt alle Pakete mit ihrer jeweiligen Version und Installationszeitpunkt an, die in dem Mandanten installiert sind.



Konfiguration

Installationsnutzer

Für die Installation über die Kommandozeile sowie für die automatische Aktualisierung von Scriptlib-Paketen wird ein Installationsnutzer benötigt. Standardmäßig wird hierfür der Nutzer `webRoot` verwendet. Falls dieser nicht vorhanden ist, wird er automatisch von otr-package angelegt.

Alternativ kann ein anderer Nutzer als Installationsnutzer festgelegt werden. Hierfür gibt es folgende Möglichkeiten:

Methode	Beschreibung
Mandanteneigenschaft	Der Nutzerlogin wird in der Mandanteneigenschaft `$packageAdmin` spezifiziert.
Umgebungsvariable	Der Nutzerlogin wird in der Umgebungsvariable `DOCUMENTS_PACKAGE_ADMIN` spezifiziert.

Hinweis: Wenn beide Methoden verwendet werden, hat die Angabe in `$packageAdmin` Vorrang.

Der Nutzer kann gesperrt werden, so dass er keine Nutzerlizenz belegt. otr-package kann ihn dann dennoch verwenden.

Build-Konfiguration

Die Erstellung der Script-XMLs kann zentral in der Datei `otr-build.json` konfiguriert werden. Dabei kann eine der Build-Konfigurationen für den Build-Prozess ausgewählt werden.

```
documents build --buildConfig build
```

In einer Build-Konfiguration können folgende Einstellungen konfiguriert werden. Sofern nicht anders angegeben, gelten die Beispielwert als Standardwerte der jeweiligen Einstellungen.

```
{
    "build": {

        // Aktivierung oder Deaktivierung der Script-Verschlüsselung (// #crypt)
        "encrypt": true,

        // Aktivierung oder Deaktivierung der Script-Komprimiierung (// #compress)
        "compress": true,

        // Erzwinge die Verschlüsselung aller Script, außer die markiert mit "//
#nocrypt"
        "forceEncrypt": false,

        // Erzwinge Komprimierung aller Scripte. Funktioniert nicht mit //#import!
        "forceCompress": false,

        // Glob-Muster für die Script-Dateien, die für beim Erstellen der
        // Import-XML eingebunden werden sollen.
        "includeSourceOnlyPattern": [
            "src/jscript/**/*.js",
            "src/**/*.xml"
        ],

        // Glob-Muster für die Scripte aus Abhängigkeiten, die beim Bauen der
        // Import-XML mit eingebunden werden sollen.
        "includeDependenciesPattern": [
            "node_modules/@otr-documents/*/src/jscript/**/*.js",
            "node_modules/@otr-documents/*/src/**/*.xml"
        ],

        // Ausgabeverzeichnis für die XML-Dateien.
        "outputDir": "build",

        // Name der XML-Dateien. Wenn nicht angegeben, dann wird der Name
        // aus der package.json verwendet.
        "xmlFileName": "beispiel",

        // Optional. Es können mehrere Build-Konfigurationen angegeben werden.
        // Mit dieser Anweisung können diese zu einer zusammengeführt werden.
```

```
        "extends": [
            "build-conf1",
            "build-conf2"
        ],

        // Optional. Name einer oder mehrerer Build-Konfigurationen, die vor
        // der Ausführung dieser Build-Konfiguration ausgeführt werden müssen.
        "needs": "build", // oder "needs": ["build", "build2", "build3", ...]

        // Aktiviert die Ersetzung von folgenden Ersetzungsmarken im den Scripten
        // * @buildDate@: Datum im Format YYYY-MM-DD
        // * @buildTimestamp@: Zeitstempel in ISO8601-Format
        // * @versionNumber@: Versionnummer aus der package.json
        // * @gitRevision@: Git-Revision letzten Commits
        "replaceVersionTags": true ,

        // Optional: Versionsnummer. Default: Versionsnummer aus der package.json
        "versionNumber": "1.0.0",
    }
}
```

Zusätzliche Ressourcen einbinden

Neben den Scripten könnnen noch weitere Ressourcen in ein SDP eingebunden werden. Hierzu ist in der Projektwurzel die Datei `otr-build-addon.json` zu erstellen. Dann wird beim Bauen des SDPs diese Dateien in das SDP eingefügt:

```
{
    "build": [
        {
            "resource": [
                {
                    // Frei wählbarerer Namen. Im SDP wird daraus ein
                    // Ordner erstellt, in dem die Dateien abgelegt
                    // werden. Empfohlen wird, diese nach den Typ
                    // zu trennen; filetypes, folders, workflows usw.
                    "name": "filetypes",
                    // Fester Wert
                    "type": "property",
                    // Filtermuster für die Dateieen der Ressource
                    // [VERSION] wird durch die Versionsnummer aus der
                    // package.json ersetzt
                    "directory": "src/filetype/[VERSION]/**/*.xml"
                }
            ]
        }
    ]
}
```

Um diese Ressourcen bei der Installation einzuspielen, sind diese im PACKAGE-Script zu referenzieren. Für das zuvor genannte Beispiel, sieht es wie folgt aus, wenn ein XML-Export mit einem Mappentypen `submission` darin abgelegt ist:

```
    updateScripts: {
        "1.0.0": [
            "filetypes/submission.xml",
            // ...
        ]
    }
```

Hier wird die XML aus den Ressourcen, die in der `otr-build-addon.json` unter dem Namen `filetypes` angegeben wurde, imporiert.

Deploy-Konfiguration

Die Erstellung der Script-XMLs kann zentral in der Datei `otr-build.json` konfiguriert werden. Dabei kann eine der Deploy-Konfigurationen für den Deploy-Prozess ausgewählt werden.

```
documents deploy --distribution --deployConfig deploy
```

In einer Deploy-Konfiguration können folgende Einstellungen konfiguriert werden. Sofern nicht anders angegeben, gelten die Beispielwert als Standardwerte der jeweiligen Einstellungen.

```
{
    "deploy": {

        // Hooks, die vor oder nach der Installation ausgeführt werden sollen
        "hooks": {
            // Hooks, die nach der Installation ausgeführt werden
            "afterImport": [
                {
                    // Name der auszuführenden Funktionen, verfügbar sind
                    // - doMaintenance (Name der Wartungsoperation)
                    // - downloadCurrentLogFile (Zielpfad)
                    // - executeScriptOnServer (Scriptname)
                    // - runCodeOnServer (Code)
                    "action": "doMaintenance",

                    // Parameter der Funktion
                    "params": ["encryptMarkedScripts"],

                    // Erwartetes Ergebnis. Wenn das Ergebnis nicht mit dem
                    // hier angegeben übereinstimmt, dann schlägt das
                    // Installieren fehl.
                    "expectedResult": "Scripts encrypted"
                }
            ],
            // Hooks werden vor der Installation ausgeführt
            "beforeImport": [
                // Siehe àfterImport`
            ]
        },

        // Optional. Wenn angegeben, wird dieser Präfix zusammen mit einer
        // generierten eindeutigen Id al Mandantnenamen verwendet
        "principalIdPrefix": "beispiel",

        // Optional. Es können mehrere Deploy-Konfigurationen angegeben werden.
        // Mit dieser Anweisung können diese zu einer zusammengeführt werden.
        "extends": [
            "deploy-extends1",
            "deploy-extends2"
```

```
        ],

        // Optional. Array von Dateien, die vor dem Import an den Server
        // geschickt werden sollen.
        "staticSources": [
            {
                "pattern": "src/templates/globalConfig/*.*",
                "globOptions": {},
                 // Werden abgelegt in /tmp/doc6_tmp/beispiel/templates/...
                "targetLocation": "beispiel/templates/"
            }
        ],

        // Optional. Name der Build-Konfiguration, die verwendet werden solle,
        // wenn `deploy` mit dem Argument `--build` aufgerufen wird.
        "buildConfig": "",

        // Optional. Vor der Installation wird die angegebene Datensicherung
        // importiert, anstatt einen neuen, leeren Mandanten zu erstellen.
        "backup": {
            "jexPath": "path/to/jex(.zip|.jex)",
            "repositoryPath": "path/to/repo.zip"
        },

        // Optional. Pfade zu Lizenzdatei(en), die vor der Installation zum
        // Server geschickt werden sollen.
        "licences": []
    }
}
```

Changelog

0.7.5

- Konfigurationseinstellung im PACKAGE-Script konnten zu Fehlern bei der Installation über das Admin-Center führen. (#72464)

v0.7.4

- In einigen Fällen sprang der Fortschrittsbalken bei der Installation sofort oder zwischendurch auf 100 %. (#67334)

v0.7.3

- Korrektur beim Aufräumen der temporären Eigenschaften nach der Ausführung von Scriptlib-Updates. (!420)

v0.7.2

- Korrektur für Anzeige in der Paketliste des Admin-Centers. (#68998)

v0.7.1

- Das Fehlschlagen der Installationsvoraussetzungen (`prerequisites`) wurde nicht korrekt behandelt. (#71060)
- Bereits installierte Pakete wurden im Ausführungsplan bei einer Paketinstallation angezeigt. (#70920)
- Beim Serverstart wurden einige Funktionen im Serverstart-Callback von otrPackage ohne Nutzerkontext ausgeführt, was zu Fehlern führte. (#71412)

v0.7.0

- Versionsgetreue Installation von Paketen. Damit werden für ein ADP bzw. die Lösung die Versionen der Lösung in exakt der Reihenfolge installiert, wie sie veröffentlicht wurden. Dies setzt voraus
  - dass am Mandanten die Eigenschaft `$PACKAGE_INSTALL_PINNED_VERSION_ENABLED` auf den Wert `true` gesetzt ist. Alternativ kann auch eine gleichnamige Umgebungsvariable gesetzt werden.
  - Die SDPs der Lösungsversionen mit ihren Ausführungsplänen gebaut wurden. Fehlen die Ausführungsplänen in den SDPs wird normal wie bisher installiert. (#67952)
- Der Backport wurden vereinfacht. In ein ADP kann nun eine Zip-Datei `backport.zip` gelegt werden. Ist diese vorhanden, wird diese nach entpacken der SDPs aus dem ADP entpackt und die darin enthaltenen Dateien in Zielordner der SDPs kopiert. Dadurch werden die XML-Dateien aus den XMLs überschrieben, bevor diese in den Mandanten importiert werden. Die Namen der Ordner in der `backport.zip` müssen dabei den Namen und der Version des jeweiligen SPDs entsprechen, deren XML-Dateien überschrieben werden sollen. (#64558)
- Post-Import-Action-Scripte in hochgeladenen XML-Dateien werden nun ausgeführt. Sowohl beim Hochladen von XMLs über die Paketverwaltung des Admin-Centers, als auch über die reguläre Installation aus einem SDP. (#67472)

- Neue Funktion `executeUpdateScriptIfMain()`, die in Update-Scripts verwendet werden kann, um Update-Scripte direkt auszuführen. (#54214)
- Die Dauer der einzelnen Installationsschritte und die Gesamtdauer der Installation werden im Installationsbericht im Property `duration` als Millisekunden erfasst. (#62220)

v0.6.0

- Hochgeladene Installationsdateien werden als Custom-Property-Blobs temporär gespeichert. Falls während der Installation die zugrunde liegende VM bzw. Container neu gestartet wird und der Ordner mit den temporären Dateien gelöscht wird, werden diese Dateien wieder automatisch aus dem Custom-Property-Blobs wieder zur Verfügung gestellt. (#67575)

v0.5.1

- Admin-Center berücksichtigte nicht den Status für den Server-Absturz. Die Installation konnte daher nicht fortgesetzt werden. (#63473)

v0.5.0

- Auswertung "vorgefertigter" Ausführungspläne, die in SDP mit ausgeliefert werden können (siehe otr-build v2.43.0). Diese werden bei der Installation eines ADPs berücksichtitigt, so dass die einzelnen Versionen der Lösung, die in dem ADP enthalten ist, in der genau Reihenfolge installiert wird, wie sie zum Zeitpunkt der Freigabe in den vorgefertigen Ausführungsplänen definiert wurden. Das schließt die Versionen der Abhängigkeiten mit ein. (#59962)
- Überarbeitung der Abhängigkeitsbaum-Algorithmus, so dass nicht mehr bei jeder neuen Abhängigkeit der gesamte Abhängigkeitsbaum durchlaufen wird. (#66553)
- SDPs konnten nicht über das Admin Center installiert werden. (#66013)
- XML-Exporte können nun über die Paketverwaltung im Admin Center importiert werden. Deren Import wird ebenfalls in den Installationsberichten vermerkt. (#65928)
- Admin-Center-Module werden von otrPackage nicht aktualisiert. (#64057)
- Mappen-Ändern war nach Serverabsturz nicht vollständig fortführbar. (#63473)

v0.4.9

- Korrektur beim Aufbau des Abhängigkeitsbaum. In einigen Fällen wurden Abhängigkeiten mehrfach in den Abhängigkeitsbaum eingetragen, was die Installation verlangsamte. (#66553)

v0.4.8

- Funktionen `getPreviousVersionName()` und `isPreviousVersionAvailable()` lieferten bei ADPs mit mehreren Versionen nicht mehr die Angabe von vor Beginn der Installation. (#64675)

- Korrektur der Typings für den Kontext von `execute()` und `postHandling()` der Update-Scripte. (#64675)

v0.4.7

- Dokumentierten Default-Verhalten des `keepUpdateScripts`-Flag, so dass es mit dem fehlerhaften Verhalten vor v0.4.6 übereinstimmt. Per Default werden Update-Scripte nach der Installation nicht gelöscht. (#64370)

v0.4.6

- Ergänzung der Korrektur für den Status "running" für Korrektur aus v0.4.5. (#64004)
- Update-Scripte wurden nach erfolgreicher Installation nicht gelöscht, auch wenn dies konfiguriert war. (#64370)

v0.4.5

- Der Fortschrittsbalken erzeugte Fehlermeldungen im Server-Log, wenn noch kein Fortschritt protokolliert wurde. Die Funktionalität des Fortschrittsbalkens war dadurch aber nicht beeinträchtigt. (#64004)

v0.4.4

- Fehler beim Entpacken von ADP/SPD-Paketen wurden verschluckt.
- Wurde eine neuere Version einer Scriptlib-Bibliothek in einem Mandanten installiert, so wurde immer die ältere Version aus dem Scriptlib-Ordner installiert. (#59988)

v0.4.3

- Fortsetzen fehlgeschlagener Installationen im Admin-Center. (#62777)

v0.4.2

- Korrekturen in den Sprachproperties. (#62777)

v0.4.1

- Korrekturen im Admin-Center, so dass fehlende Datumswerte in Installationsberichten keine Fehler auslösen. (#62777)

v0.4.0

- Installationsstatus war nicht sofort "running", nach dem eine Installation gestartet wurde. Das erfolgte erst, nachdem der Listener aufgerufen wurde. Für diesen Zweck wurden interne API-Funktionen ergänzt. (#62777)
  - `initInstallationProgress()`
  - `setInstallationProgress()`
- Englische Sprachproperties für das Admin-Center und verschiedene, kleinere Verbesserung der Integration selbst. (#62777)

v0.3.1

- Kleinere Korrekturen der Admin-Center-Integration. (#62777)

v0.3.0

- Integration in das Admin-Center von documentsOS, so dass ADP-Pakete direkt aus dem Admin-Center installiert werden können. Die bisherigen Wizard sind damit veraltet und werden nicht mehr weitergepflegt. In zukünfigen Version werden die Scripte auch entfernt. (#62777)
- Neue API-Funktionen, um Installationsstatus und -berichte abzufragen (#62884):
  - `getInstallationReports()`
  - `getInstallationReportByIndex()`
  - `findInstallationReport()`
  - `findCurrentInstallation()`
- Neue API-Funktionen zum Steuern des Installationsprozesses (#62777):
  - `abortInstallation()`
  - `augmentExecutionPlanWithConfigValues()`
  - `continueInstallation()`
  - `findErrorInExecutionPlan()`
  - `getConfigParameter()`
  - `getConfigValueKey()`
  - `getInstallationStatus()`
- Mandant wird während der Installation nur noch dann neugeladen, wenn mit einer XML Custom Properties importiert wurden. Das Neuladen setzt die Sprachproperties zurück, wodurch die neu geladen werden. Das bremste Funktionen, die auf die Sprachproperties während er Installation zugreifen. (#63294)
- Datumswerte werden in Installationsberichten und Versionshistorie einheitlich in im Format ISO 8601 gespeichert. Alte Einträge werden beim ersten Serverneustart nach dem Update auf das ISO-8601-Format migriert. (#62784)
- Mappen-Ändern wurde für globale Update-Skripte nicht ausgeführt. (#62589)
- Fehler im Ausführungsplan (bei der Installation) werden als Exception geworfen. (#62053)

v0.2.0

- Installation von Modulen wurde für Cloud/Komandozeile vereinheitlicht. (#57510)
- Neue Funktion `getExecutionPlanAndDependenciesFromADP()`, um den Ausführungsplan eines Moduls zu ermitteln. Dazu wird auch der 'minimale' Abhängigkeitsbaum erstellt. (#57510)
- Neue Funktion `deployLocalADP()`. Ermöglicht das Deployen einen Module mit Hilfe der Pfadeingabe des `.adp`-Pakets auf dem System des Benutzer/des Service. (#57510)
- Neue Funktion `getAllOtrPackageProperties()`: Gibt alle Eigenschaften, die während einer Modul-Installation am Mandant hinterlegt wurden (History, Errors, ...) zurück. (#57510)
- Neue Funktion `otrPackageUtils.isScriptLibModule()`: Ermöglicht die Suche in den ScriptLibs nach der eingegebene `Scriptname`. Wenn diese gefunden wird, gibt die Funktion `true` zurück, sonst `false`. Die Suche erfolgt mittels `otris.tools.getScriptNamesFromMapFiles`. (#57510)
- Stürzt während der Installations eines Pakets der Server ab, so wird nun die Installation an der Stelle forgesetzt. Bisher wurde von vorne begonnen. (#60320)
- Wurde die Paket-Deklaration per `require()` eingebunden, funktionierten nicht alle Funktionen, wenn dabei ein Destructive Assignment verwendet wurde (`const { isActive } = require("Example_PACKAGE");`). (!147)
- Update Center aus AdditionalSetttings-Menü entfernt, da nur ein Prototyp. (!167)
- Allgemeine Verbesserung der Handhabung von Server-Abstürzen während der Installation. (#59267)
- Allgemeine Verbesserung beim Wiederaufsetzen der Installation nach einem Abbruch durch einen Fehler. (#59267)
- Automatische Prüfung, ob die verwendete Version von Scriptlib-Modulen in einem Mandanten jeweils mit der Version übereinstimmt, die mit der verwendeten Serverversion ausgeliefert wurde. Ist das nicht der Fall, wird bei der Installation eines ADP eine Warnung in das Server-Log geschrieben. (#60321)

v0.1.36

- Wurde die Paket-Deklaration per `require()` eingebunden, funktionierten nicht alle Funktionen, wenn dabei ein Destructive Assignment verwendet wurde (`const { isActive } = require("Example_PACKAGE");`). (#60363)

v0.1.36

- Wurde die Paket-Deklaration per `require()` eingebunden, funktionierten nicht alle Funktionen, wenn dabei ein Destructive Assignment verwendet wurde (`const { isActive } = require("Example_PACKAGE");`). (#60363)

v0.1.35

- Neues Property `submitFileChangeMode` in der Paketdeklaration, welches steuert, wann Mappenändern und damit die Post-Handlings ausgeführt werden. (#61954) Unterstützt werden die Modi:
  - `local`: Jeweils nach Installations einer Version eines Pakets (Pro SDP)
  - `global`: Gebündelt nach Installation aller Versionen aller Pakete (Einmal für das gesamte ADP) Durch setzen der globalen Documents-Eigenschaft `$packageSubmitFileChangeMode` kann diese Einstellung in Paketen überschrieben werden.

v0.1.34

- Das Script "otrPackageManager" wird ab 5.0i HF7 bzw. 6.0.2 automatisch beim Serverstart aus dem Mandanten gelöscht, wenn vorhanden. (#62058)

v0.1.33

Technisches Release aufgrund von Problemem mit otr-build bzw. dem otrisbook.

v0.1.32

- Beim Installieren in der Cloud wurden Fehler bei der Installation nicht an das Cloud Control weitergegeben. (#62053)
- Bei Neustart einer fehlgeschlagenen Installation in der Cloud schlug mit einem Fehler fehl. (#62053)
- Das Script "otrPkgDistributionInCloud" wird ab 5.0i HF7 bzw. 6.0.2 automatisch beim Serverstart aus dem Mandanten gelöscht, wenn vorhanden. (#62058)

v0.1.31

- Filterung bereits installierter Versionen korrigiert. Es wurde nicht die Semver-Sortierung berücksichtigt, so dass z. B. v3.7.0 als neuer als v3.10.0 angesehen wurde. Das betraf ebenso die Erzeugung der initialen Versionshistorie, wodurch dann falsche Versionen darin enthalten waren. (#61573)
- Entfernung unnötiger Log-Ausgaben. Die Log-Datein wurde bei umfangreichen Paketen sonst zu groß.

v0.1.30

- Allow-Client-Definitionen für Script-Ausführung im Web ergänzt.

v0.1.29

- Bestimmung der neusten Version eines SDPs bei der Installation korrigiert. Bisher konntne dadurch nur Patch-Version bis x.y.9 installiert werden. x.y.10 usw. wurden ignoriert. (#60815)

v0.1.28

- Ermittelte Abhängigkeiten werden im Custom Property `PackageDependency` zwischengespeichert. (#59989)

v0.1.27

- `roles` in Paket-Deklaration sind auch für Basic-Pakete erlaubt. (!136)
- Optimierung beim Auslesen des Modul-Properties. (!137)

v0.1.26

- `configurationType` in der Paket-Deklaration zu `type` mit den folgenden Werten umbenannt (#60102):
  - `basic`: Standard-Paket, die nicht im Admin-Center eingebunden sind
  - `extension`: Paket, das in das Admin-Center eingebunden werden soll

v0.1.25

- Erweiterung der Paket-Deklaration um Eigenschaften und Funktionen, um das Paket in das Admin-Center einzubinden. Alle Funktionen sind optional. Im Fall von `isActivate`, `activate` und `deactivate` gibt es Default-Implementierungen. (#59796)

```
module.exports = require("otrPackage").declarePackage({
  name: "example",
  version: "1.0.0",
  configurationType: "none" | "tile", /* tile = Kachel im Admin-Center */
  isActive: () => { /* Abfrage, ob Paket aktiv */ },
  activate: (options) => { /* Aktivierung des Pakets */ },
  deactivate: (options) => { /* Deaktivierung des Pakets */ },
  actions: () => { /* Abfrage der Aktionen */ },
  callAction: (name) => { /* Aufruf einer Aktion */ },
  roles: () => { /* Abfrage der Rollen des Pakets */ }
});
```

- Rollback-Mechanismus für PACKAGE-Scripte. Tritt beim Importieren der PACKAGE-Scripte ein Fehler auf, weil z. B. die `prerequisites` nicht erfüllt sind, werden alle Scripte der Kategorie

`init`, und damit min. alle PACKAGE-Scripte wiederhergestellt. Der Mandant ist danach wieder im Originalzustand. (#59208)

v0.1.24

- Erweiterung der Paket-Deklaration um eine Funktion zum Prüfen von Voraussetzungen. Die Funktion wird direkt nach dem Import des Package-Deployment-Descriptors ausgeführt. Wirft diese eine Exception oder liefert einen Array mit Strings (Fehlermeldungen) zurück, wird die Installation sofort abgebrochen, und die Fehlermeldungen ausgegeben. (#59207)

```
module.exports = require("otrPackage").declarePackage({
  name: "example",
  version: "1.0.0",
  prerequisites: () => {
    const unfulfilled = [];
    if (!context.hasPEMModule(context.PEM_MODULE_GADGETS)) {
      unfulfilled.push("Gadget license is missing");
    }
    if (!context.hasPEMModule(context.PEM_MODULE_DROP)) {
      unfulfilled.push("pf:MISSING_DROP_LICENSE");
    }
  // ...
  return unfulfilled;
}
});
```

- Bei Aktualisierungen durch das Auto-Start-Script des Servers wird der Package-Admin-Nutzer nicht mehr angelegt. Fehlt dieser, es sollen aber Pakete aktualisiert werden, wird nun eine Exception geworfen. (#58745)

v0.1.23

- Neue Funktionen am Package-Context (#58025):
  - `getPreviousVersionName()`: Liefert die Versionsnummer der aktuell installierten Version des aktuellen Pakets zurück.
  - `isPrewviousVersionAvailable()`: `true`, wenn bereits eine Vorgängerversion des aktuellen Pakets installiert wurde.
- Der Rückgabewert der execute- und der postHandling-Funktion eines Update-Scripts ist nun optional. Er kann zudem verwendet werden, um Warnungen in den Installationsbericht einzufügen. (#56147, #58014)
- Korrektur bei der Installation eines ADPs in der Cloud. (#58007)
- Beim Entpacken von SPDs und ADPs wurde der Fehlercode nicht ausgewertet, so dass Fehler beim Entpacken unbemerkt blieben. (#57954)

- Update-Scripte mit Punkt im Namen und Dateierweiterung wie `script.install.js` wurden nicht ausgeführt. (#56560)
- Korrektur bei der Installation eines ADP in der Cloud. (#58007)
- Berücksichtigung des Breaking Change beim Default-Zeichensatz der `File`-Klasse im Scripting in Documents 6, weswegen XML-Importe kaputt gehen konnte, wenn die XML Zeichen enthielt, die vom Default-Zeichensatz des unterliegenden Betriebssystems nich unterstützt wurden. Betraf nur Windows. (#58527)
- Nach dem Import einer XML wird automatisch der Mandant neu geladen, so dass die verschiedenen Caches zurückgesetzt werden. (#58753)
- ADPs von Projekten mit Leerzeichen im Namen können nicht installiert werden. (#56365)
- Wird eine Update-XML für einen Mappentyp während der Installation importiert, dann wird der Mappentyp automatisch im `submitFileChanges` eingetragen. (#56153)
- Für Zielversionen kann die benötigte Serverversion im Package-Script definiert werden:

  ```
  latestServerVersions: [{
    needsServerVersion: 2501,
    version: "1.0.0"
  }, {
    needsServerVersion: 2530,
    version: "1.1.0"
  }]
  ```

  Hier benötigt Version v1.0.0 die Serverversion 2501 oder höher, Version v1.1.0 Serverversion 2530 oder höher. Wird v1.1.0 z. B. auf einer 2500 eingespielt, dann wird bei Beginn der Installation ein Fehler geworfen. (#54897)
- Importe großer XML-Dateien konnten zu Abbrüchen der Installation führen. (#54756)
- Das automatische Löschen des Update-Scripte kann durch den neuen Schalter `keepUpdateScripts` im Package-Script deaktiviert werden. (#54695)
- Im Package-Script kann nun eine Sperrliste für Scripte im Mandanten definiert werden. Ist die Serverversion gleich oder höher als angegeben, wird das Script bei der Aktualisierung automatisch gelöscht. (#54491) Im Folgenden wird das Script `util.otrPerfmon` nur dann gelöscht, wenn es min. der Server min. eine 2322 ist. Andernfalls nicht:

  ```
  scriptBlacklist: [{
    sinceServerVersion: "2322",
    scriptBlacklist: [
      "util.otrPerfmon"
    ]
  }, {
    app_version: "4.0.1",
    scriptBlacklist: []
  }]
  ```

- Die Sperrliste in der Paketdeklaration (blackListScripts) hatte sich geändert und die Abwärtskompatibilität gebrochen. Diese wurde wieder hergestellt. (#58208)

- ADP-Installation schlug kann wegen eines fehlenden `incVersion` fehlschlagen, wenn die erweiterte Angabe der Update-Scripte verwendet wurde. (#54292)

v0.1.22

- Für Zielversionen kann die benötigte Serverversion im Package-Script definiert werden (#54897):

  ```
  latestServerVersions: [{
      needsServerVersion: 2501,
      version: "1.0.0"
  }, {
      needsServerVersion: 2530,
      version: "1.1.0"
  }]
  ```

  Hier benötigt Version v1.0.0 die Serverversion 2501 oder höher, Version v1.1.0 Serverversion 2530 oder höher. Wird v1.1.0 z. B. auf einer 2500 eingespielt, dann wird bei Beginn der Installation ein Fehler geworfen.
- Importe großer XML-Dateien konnten zu Abbrüchen der Installation führen. (#54756)
- Das automatische Löschen des Update-Scripte kann durch den neuen Schalter `keepUpdateScripts` im Package-Script deaktiviert werden. (#54695)
- Im Package-Script kann nun eine Sperrliste für Scripte im Mandanten definiert werden. Ist die Serverversion gleich oder höher als angegeben, wird das Script bei der Aktualisierung automatisch gelöscht. (#54491) Im Folgenden wird das Script `util.otrPerfmon` nur dann gelöscht, wenn es min. der Server min. eine 2322 ist. Andernfalls nicht:

  ```
  scriptBlacklist: [{
    sinceServerVersion: "2322",
    scriptBlacklist: [
      "util.otrPerfmon"
    ]
  }, {
    app_version: "4.0.1",
    scriptBlacklist: []
  }]
  ```

- Wird eine Update-XML für einen Mappentyp während der Installation importiert, dann wird der Mappentyp automatisch im `submitFileChanges` eingetragen. (#56153)
- ADP-Installation schlug kann wegen eines fehlenden `incVersion` fehlschlagen, wenn die erweiterte Angabe der Update-Scripte verwendet wurde. (#54292)
- Für den webRoot-Nutzer kann ein anderer Nutzer konfiguriert werden. Entweder durch die Umgebungsvariable `DOCUMENTS_PACKAGE_ADMIN` oder die Mandanteneigenschaft $packageAdmin. Sind beide definiert, dann greift `$packageAdmin`. (#56561)

- Wurde ein ADP ein zweites Mal über die Konsole installiert, dann kam eine Fehlermeldung. (#57520)
- Der webRoot-Nutzer, der von otr-package automatisch angelegt wird, war per Default in Documents-Listen sichtbar. (#56947)